SEDust: User Manual

Abstract

This manual describes the model-agnostic dust thermal-emission library under `SEDust/sed/`. The library computes the infrared emission spectrum of a dust mixture for an arbitrary local radiation field, and is intended to be linked into a Fortran 3D radiative-transfer (RT) code. It handles several dust models as peers — the HD23 *astrodust*+PAH model, the Draine & Li (2007) silicate+carbonaceous model, the Zubko, Dwek & Arendt (2004) BARE-GR-S model, and arbitrary file-defined models — through one object type and one emission call. The numerically validated stochastic-heating solver is reused unchanged; the library is an additive layer on top of it.

Contents

1. Overview

A *dust model* is represented by a single derived type, `dust_model_t`. You build it once (per RT run), then call `dust_emission` once per RT cell with that cell's local mean intensity:

```
use dust_lib
type(dust_model_t) :: m
```

```
real(wp), allocatable :: J_lam(:), lamI_total(:)

call build_astrodust(m, qtab, sizedist, 200, 2.7_wp, 5.0e3_wp) ! once
allocate(J_lam(m%NLAM), lamI_total(m%NLAM))
do icell = 1, ncells
   ! ... assemble J_lam(:) (mean intensity on the grid m%lam) for this cell ...
   call dust_emission(m, J_lam, lamI_total) ! per cell
   ! ... lamI_total(:) is lambda*I_lambda / N_H for the cell ...
end do
```

Design. The library wraps the existing, FIR-validated solver core (`sed_grain_loop`, `calc_Teq`, `calc_P`) without modifying it. A model is a set of grain *populations* (one per species and charge state), each carrying its own size distribution, absorption cross sections, enthalpy, and Planck-integral tables. `dust_emission` loops the populations through the solver and sums their emission into named output *channels*.

One active model at a time. The module-global grids (`lam`, `T_first`, ...) hold the *currently built* model's working set. Calling `build_*` makes that model active. This is sufficient for an RT run that uses a single dust model; to switch models, rebuild.

2. Building the library

From `SEDust/sed/`:

```
make libsedust.a # the library archive (link this into your RT code)
```

The compiler is `gfortran` with `-O3 -march=native -fopenmp` (edit the `Makefile` for ifort). `libsedust.a` is a static archive of all library objects; the Fortran `.mod` files are written alongside it in `SEDust/sed/`.

3. Models and builders

Each builder fills a `dust_model_t` and defines the model's output channels (Table 1).

builder	model	channels	notes
`build_astrodust`	HD23 astrodust+PAH	`AD`, `PAH`	N_C=417, D16 graphite
`build_dl07`	Draine & Li 2007	`SIL`, `CARB`	N_C=470, D03 graphite
`build_zubko`	Zubko (ZDA 2004) BARE-GR-S	`PAH`, `GRA`, `SIL`	neutral PAH only
`build_from_files`	file-defined	`CH1`, `CH2`, ...	generic loader

Table 1: Built-in model builders. Each sets the model's own optics/abundance parameters internally.

Signatures:

```
call build_astrodust(m, qtable_path, sizedist_path, NT, T_lo, T_hi [, status])
call build_dl07 (m, qtable_path, sizedist_path, sd_index, U, NT, T_lo, T_hi [, status])
call build_zubko (m, config_path, data_dir, NT, T_lo, T_hi [, status])
call build_from_files(m, descriptor_path, data_dir, NT, T_lo, T_hi [, status])
```

- `NT`, `T_lo`, `T_hi`: number and range [K] of the internal temperature grid (typical `200, 2.7, 5000`).
- `sd_index` (DL07): WD01 size-distribution index — `7` = MW R_V=3.1, $q_{\rm PAH}$=4.58%; `29/30/31` = LMC2; `32` = SMC.
- `U` (DL07): ISRF intensity scaling (the DL07 reference uses U=1).
- `data_dir` (Zubko / files): directory holding the data files, e.g. `data/zubko/`.
- `status` (optional, all builders): error report; see §4.5. When present, a missing or malformed input file is reported through it and the model is not built, so an RT host can recover.

4. Computing emission

4.1 Single-cell exact solve

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status])
```

- `J_lam(m%NLAM)`: the local *mean intensity* on the grid `m%lam` [μm]. For a scaled Mathis ISRF use `call J_Mathis(U, m%lam, J_lam)` from module `radfield`.
- `lamI_total(m%NLAM)`: the summed SED, $\lambda I_\lambda / N_{\rm H}$ [erg s^{-1} cm^{-2} sr^{-1} H^{-1}] — the HD23 convention.
- `lamI_chan(m%NLAM, m%n_channel)` (optional): the SED of each channel.
- `status` (optional): error report; see §4.5.

4.2 Solver choice (`m%stoch_method`)

The field `m%stoch_method` selects how each grain is heated (Table 2); `dust_emission` honors it.

4.3 Equilibrium-temperature options

Two options replace the full stochastic solve when an equilibrium approximation is wanted:

(1) Equilibrium temperature for each grain type and size. Set `m%stoch_method = 'equil'` and call `dust_emission`. Every grain is placed at its own equilibrium temperature $T_{\rm eq}$, computed from *that grain's* absorption cross section, and emits $C_{\rm abs} B_\lambda(T_{\rm eq})$. No stochastic excursions are computed.

m%stoch_method	method	note
'heuristic'	GD look-ahead T-window narrowing	**default**; fastest, serial
'draine'	Guhathakurta & Draine (1989)	reference GD
'qm'	energy-space transition matrix (thermal-discrete)	most detailed
'equil'	single-grain equilibrium (no stochastic)	see §4.3

Table 2: Stochastic-heating solver options. The first three resolve the grain temperature *distribution* (PAH/NIR features); 'equil' forces equilibrium.

(2) A single equilibrium temperature for the whole model.

```
call dust_emission_single_teq(m, J_lam, lamI_total [, Teq_out])
```

All dust shares one temperature, found from the type- and size-integrated (dn-weighted) total absorption cross section $C^{\rm tot}_{\rm abs}(\lambda) = \sum_{\rm pop}\sum_a \mathrm{d}n(a)C_{\rm abs}(\lambda,a)$. The single $T_{\rm eq}$ satisfies $\int C^{\rm tot}_{\rm abs} J\,\mathrm{d}\lambda = \int C^{\rm tot}_{\rm abs} B(T_{\rm eq})\,\mathrm{d}\lambda$, and the emission is $C^{\rm tot}_{\rm abs} B_\lambda(T_{\rm eq})$. The optional Teq_out returns that temperature.
Both options conserve energy (the emitted bolometric power equals the absorbed power); they differ from the stochastic solve only in the SED *shape* — a smooth modified blackbody rather than one carrying PAH/NIR features.

4.4 Precomputed table + interpolation

When the field in each cell is a fixed spectral *shape* scaled by an intensity U, precompute a table once and interpolate per cell:

```
type(dust_emis_table_t) :: tab
call dust_build_table(m, J_ref, U_grid, tab [, status]) ! once
call dust_emission_interp(tab, U, lamI_total [, lamI_chan] [, status]) ! per cell
```

J_ref(m%NLAM) is the reference field shape (at U=1) and U_grid(:) an ascending intensity grid. The table is built from exact solves, so it is exact at the grid points; interpolation is log–log in U. The stored emissivities are floored at 10^{-300} before the log, so a grid node whose true value is 0 comes back as 10^{-300} rather than exactly 0. Both calls take an optional final status (§4.5).

Caveat. Stochastic emission is a *nonlinear* functional of the full $J(\lambda)$, not of a scalar. The table is valid only when the cell field is $U \times$ (fixed $J_{\rm ref}$). For cells whose field *shape* departs from $J_{\rm ref}$ (e.g. hardened spectra near hot stars), use the cell-by-cell exact dust_emission.

4.5 Optional error status

The builders and the emission/table calls each accept an optional final status argument (integer, intent(out)). When it is present, an invalid model, argument, or input file is reported through it (status = 0 on success, nonzero on failure) and the process keeps running, so a 3D RT host can recover; when it is omitted, the same condition prints

a message and stops the run, which the CLI drivers rely on. For the builders the nonzero value only names the failing stage: the contract is simply 0 = built, nonzero = build failed.

- dust_emission: 1 unknown stoch_method; 2 'qm' selected but a population's radii are unset.
- dust_build_table: 1 size(J_ref) ≠ m%NLAM; 2 size(U_grid) < 2; 3 U_grid not positive and strictly increasing.
- dust_emission_interp: 1 U ≤ 0; 2 size(lamI_total) ≠ tab%NLAM; 3 lamI_chan present but not (tab%NLAM, tab%n_channel).
- build_astrodust, build_dl07: 1 Q-table load failed; 2 size-distribution load failed.
- build_zubko: 1 config read; 2 fewer than 3 components; 3 a component's optics read; 4 grid inconsistent; 5 a component's calorimetry read failed.
- build_from_files: 1 descriptor open; 2 too many pop: lines; 3 invalid channel; 4 no pop: lines; 5 optics read; 6 grid inconsistent; 7 size-distribution read; 8 calorimetry read failed.

5. Computing extinction

dust_extinction is the extinction counterpart of dust_emission: a transfer code takes its opacity from the same model object, and on the same wavelength grid m%lam, as its emission.

```
call dust_extinction(m, Cext, Cabs, Csca [, gbar] [, status])
```

- Cext, Cabs, Csca (m%NLAM): extinction, absorption and scattering cross sections per H atom [cm^2/H], with $C_{\rm ext} = C_{\rm abs} + C_{\rm sca}$.
- gbar(m%NLAM) (optional): the scattering-weighted asymmetry $\langle\cos\theta\rangle$, and 0 where nothing scatters.
- status (optional): error report; see §4.5. It is 0 on success and 1 if an output array is not of size m%NLAM.

Each output is the plain binned sum over every population of the model, since $\mathrm{d}n(a)$ already carries the bin width:

$$C_{\rm abs}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm abs}(\lambda,a),$$

$$C_{\rm sca}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm sca}(\lambda,a),$$

$$\langle\cos\theta\rangle = \frac{\sum \mathrm{d}n\,C_{\rm sca}\,g}{\sum \mathrm{d}n\,C_{\rm sca}}.$$

A population that carries no scattering optics contributes zero to those terms. In the astrodust model the PAHs are exactly that case: they enter through absorption only, and the astrodust grains carry all the scattering and the asymmetry. The values agree with the corresponding columns of data/kext_astrodust_MW.dat to the precision with which that file is written (the cross sections to a few parts in 10^8; the albedo and $\langle\cos\theta\rangle$ to the file's six written decimals), because both evaluate the same size integral over the same optics. The difference is that the accessor returns the result on m%lam directly, so a host using its own wavelength grid need not interpolate off the grid the driver happened to write.

Astrodust only for scattering. `build_dl07`, `build_zubko` and `build_from_files` do not populate the extinction-side scattering optics, so for those models `Csca` and `gbar` come back as zeros while `Cabs` (and hence `Cext`) is still the correct absorption sum.

6. The model object and accessors

```
type :: dust_model_t
   character(len=32) :: name
   integer :: NA, NLAM, NT
   real(wp), allocatable :: lam(:) ! (NLAM) [um]
   real(wp), allocatable :: T_first(:) ! (NT) [K]
   type(grain_pop_t), allocatable :: pops(:)
   integer :: n_channel
   character(len=16), allocatable :: channel_name(:)
   logical :: use_induced_emission ! default .false.
   character(len=16) :: stoch_method ! default 'heuristic'
   logical :: verbose ! default .false.
end type
```

Diagnostics. `m%verbose` (default `.false.`) keeps the library solve path silent; set it `.true.` to print the same solver diagnostics as the CLI drivers.

Convenience accessors (all in `dust_lib`):

```
n = dust_nlam(m) ! number of wavelengths
nc = dust_n_channel(m) ! number of output channels
lam = dust_lambda(m) ! allocatable copy of the wavelength grid [um]
nm = dust_channel_name(m, ic) ! name of channel ic
```

7. The generic file loader

`build_from_files` reads a small text *descriptor* and assembles a model from data files, with no code changes. One `pop:` line per population:

```
# descriptor.txt
name = mymodel
# pop: <grain_type> <channel> <optics> <dnda_table> <calorimetry> <rho(0=use file)>
pop: pah 1 PAH_28_1201_neu.dat SzDist_PAH.dat Graphitic_Calorimetry.dat 0.0
pop: gra 2 Gra_121_1201.dat SzDist_Gra.dat Graphitic_Calorimetry.dat 0.0
pop: sil 3 suvSil_121_1201.dat SzDist_Sil.dat Silicate_Calorimetry.dat 0.0
```

- `grain_type`: `'sil'`, `'pah'`, or `'gra'` (selects the heat-capacity/mode model used by the `'qm'` solver; ignored by the GD solvers, which use the supplied enthalpy directly).
- `channel`: integer output channel ($1 \ldots$); populations sharing a channel are summed.
- `optics`: a DustEM/Zubko Q-table ($Q_{\rm abs}$ per radius and wavelength); $C_{\rm abs} = Q_{\rm abs}\pi a^2$.
- `dnda_table`: a 2-column table a [μm] and $\mathrm{d}n/\mathrm{d}a$ [cm^{-1} H^{-1}].
- `calorimetry`: a specific-enthalpy table $u(T)$ [erg/g]; $H(T,a) = u(T)\,\rho\,\frac{4\pi}{3}a^3$.
- `rho`: grain density [g cm^{-3}]; `0.0` uses the value in the optics-file header.

All files are sought under `data_dir`. The coded builders (`build_astrodust/dl07/zubko`) are the recommended path for the built-in models; `build_from_files` is for adding new data-defined models.

8. Linking into a 3D RT code

Compile your RT source against `libsedust.a` with the `.mod` search path pointing at `SEDust/sed/`:

```
gfortran -I<sed> my_rt.f90 <sed>/libsedust.a -fopenmp -o my_rt.x
```

A complete minimal example is provided in `SEDust/sed/rt_example/use_dustlib.f90`: it builds a model, computes emission for one field, and prints the SED peak. No `ISO_C_BINDING` is needed — the RT code simply uses the Fortran module `dust_lib`.

Threading. `m` is read-only during `dust_emission`; all scratch is local. The solver uses OpenMP internally over grains. If your RT parallelizes over cells, take care to avoid oversubscription (nested parallelism); the default `'heuristic'` solver is serial per call, which suits an outer cell-level parallel loop.

9. Units and conventions

- Wavelengths `lam` are in μm throughout; cross sections in cm^2.
- `J_lam` is a mean intensity with the same units as the Planck function B_λ (specific intensity per steradian). The library's output $\lambda I_\lambda / N_{\rm H}$ is then in erg s^{-1} cm^{-2} sr^{-1} H^{-1}.
- The CMB (2.725 K) is included in the heating field by `J_Mathis`.
- The induced-/stimulated-emission factor is off by default (the output is *net* emission, matching the HD23 and DL07spec released SEDs).

10. Data files

Model data lives under `SEDust/data/`:

- astrodust / DL07: the T-matrix Q-table (`../tmatrix/output/q_astrodust_*.dat`) and the HD23 release size distribution (`../data/release/size_distribution.dat`).
- Zubko: `../data/zubko/` — the ZDA config/formula, the tabulated size distributions, the DustEM Q-tables, and the calorimetry tables from the Camps et al. 2015 benchmark; a ready descriptor is `../data/zubko/zubko_descriptor.txt`.